



Automatic Graders

FOR PROGRAMMING CLASSES

by JACK HOLLINGSWORTH*

Fifteen months ago the first version of an "automatic grader" was tried with a group of twenty students taking a formal course in programming. The first group of twenty programs took only five minutes on the computer (an IBM 650). With such a satisfactory beginning, the grader was then used for the entire course with this group of students and have been used at Rensselaer ever since. For all exercises, the average time spent on the computer has run from half a minute to a minute for each student. In general only an eighth as much computer time is required when the grader is used as is required when each student is expected to run his own program, probably less than a third as much staff time, and considerably less student time. The grader easily justifies itself on economic grounds. It accomplishes more than savings in time and money; it makes possible the teaching of programming to large numbers of students. This spring we had 80 students taking a full semester course in programming; over 120 are expected next spring. We could not accommodate such numbers without the use of the grader. Even though the grader makes the teaching of programming to large numbers of students possible and economically feasible, a most serious question remains, how well did the students learn? After fifteen months, our experience leads us to believe that students learn programming not only as well but probably better than they did under the method we did use—laboratory groups of four or five students. They are not as skilled in machine operation, however, since they get only a brief introduction to it late in the course. After learning programming, very little time is needed for each student to become at least an adequate machine operator. Students seem to like the grader and are not reluctant to suggest improvements!

Typical Use

Let us consider a typical day when class is in session. At the beginning of the period, grader results are returned. At the end of the period, each student hands in any new programs he has written and corrections to old ones. No effort is made to keep the class together and each student is allowed to work on several programs at one time if he wishes. Following the class period, these new programs and corrections are keypunched. After keypunching, the corrections are filed with the old programs and all the

programs, old and new, are arranged by exercises. We are still doing considerable hand filing of punched cards at this stage. This large deck of cards which includes the grader program is then run on the computer. Our largest single run has been 106 student programs covering 9 different exercises.

As each student program is run, the grader punches an identification card containing the student's name, exercise number and the date. The grader then proceeds to store in appropriate locations the variables for a pass and then makes a pass with the student program. If the computer stops, say on overflow, reentry is made manually to the grader at a designated location, the grader punches an "overflow" card, and follows a predetermined checkout scheme. During early exercises, the grader traces the student program; during later exercises, the student is expected to write into his programs his own checkout procedures. If the grader gets through the student program without stopping, either the results are correct or they are not. If not, the grader punches `WRONG ANSWER` and follows the predetermined checkout scheme. If the results are correct, further passes are made until either a wrong result or stop occurs or until the student program has been proved correct under a rigorous set of conditions. In this case, `PROBLEM COMPLETE` is punched, a welcome sign to the student! In either event, either the results were correct or the program is terminated when checkout is complete, and the grader proceeds to the next student. The student receives for each exercise tab sheets containing the identification, correct results, if any, and whatever checkout results were provided for by either the grader or the student.

We are currently using a second version of the grader. While both the first and second versions of the grader were carefully written and checked out, there is no satisfactory checkout possible other than trying to check student programs. We have probably checked so far some three thousand student programs using graders.

Difficulties and Limitations

While we have had considerable success using the grader, there are difficulties and limitations:

(1) The grader cannot be completely automatic. Overflow, invalid addresses, built-in stops and other effects can make the computer stop—and will usually cause it to stop again during tracing. Manual reentry is necessary.

*Computer Laboratory, Rensselaer Polytechnic Institute, Troy, New York.

(2) Student programs can modify the grader itself. While student programs can be edited and such modifications caught, we have found it to be more practical to do some editing but still take some risk of the grader getting modified.

(3) We are limited as yet to programs written in machine language or to the machine language output of a processor of symbolic language.

(4) The exercises must meet four restrictions. The location of the entrance to the student program, the location to which it must exit, the location where variables are to be found and the results to be put must all be specified. Otherwise our exercises seem to be general in nature.

Applications

We are considering the two following applications. First, a grader which will check symbolic language programs. While we may be able to conform to an existing language such as FORTRAN, we shall certainly have to write our own processor as we are now doing.

A second application is teaching programming long distance. We currently have a student who is doing the exercises by mail. He sends his programs and corrections, we send him grader results. It is still too early to know how well this procedure works.

Flow Diagram

Anyone wishing to write his own grader—and anyone with the desire to, should—will find a flow diagram his principal need. Our present grader required only 108 instructions.

Flow Diagram: (Enter 21).

1. Reset pass index
2. Has the last pass been made? If so, go to 16.
3. Store the variables and correct answers for this pass. Increase pass index by 1. Initialize lookup instructions used in 5. Enter student program.
4. Student program. If a stop occurs, go to a manual entry.
5. Check a student result against corresponding correct answer. If different, go to 9.
6. Modify instructions to compare next result with next answer.
7. Have all the results for this pass been checked? If not, go to 5.
8. Display line of variables and correct results if desired. Go to 2.
9. Set up to punch "wrong answer" and go to 14.

Manual entries:

10. If stop on overflow, set up to punch "overflow." Go to 14.
11. If stop on storage selection, set up to punch "storage selection" and go to 14.
12. If stopped because of too much time, set up to punch "too much time" and go to 14.
13. If stopped by operator decision, set up to punch "operator decision" and go to 14.
14. Status switch. If set to 1, punch cause of trouble. If set to 2, go to 17.
15. Set status switch to 2. Produce desired checkout information. If tracing is used and a stop occurs, go to a manual entry. When checkout is complete, go to 17.

16. Punch "problem complete."
17. Setup for next student. Clear drum locations. Set status switch to 1. Set name to BLANK. Go to input program.
18. Input program. Read a deck of cards. If the deck is a student program, go to 20. If the deck is data for the next exercise, go to 19.
19. Set up exercise. Set linkage between student program and grader. Compute and store initial values of the lookup instructions used in 5. Go to 18.
20. Punch identification, student name, exercise number, date. Go to 1.
21. Entrance. Stop so that date can be set on the console. Go to 18.

Acknowledgment

The grader so far has more than justified our effort and exceeded our early expectations. Credit should be given to two machine operators, Miss Rita Campana and Mrs. Marie Greenfield who have made the day-to-day operation a success and to Mr. Walter McClintock for his effort on our flexible tab panel.

* * * * *

[While there is a great deal of research being accomplished on "teaching machines", many computer educators have not realized that when teaching the use of the computer they have access to the finest "teaching machine" of all—the digital computer. This is the first of what the Editor hopes will be a fine series of papers on aspects of the education of people to use computers.—A. J. P.]

LETTERS TO THE EDITORS (*cont'd*)

if it used physically distinct elements." It may be fundamental, but the principle hasn't worked out. While it might be nice to have a high speed store without physically distinct elements, the usefulness of computers has not seemed to suffer from the lack.

The sole nod to the competence of the savages out in the colonies is given on page 340: "... magnetic tape ... has been on American machines for some time." The implied complement is then vitiated by the erroneous statement that "no simple and adequate mechanism for direct input from a keyboard on to magnetic tape has been developed ..." Neglecting, for the moment, the devices (Unitypers) which do just that, again, the lack of such devices has not hampered anyone unduly. For the bulk of machines now operating, the stages from hand keying, to cards, to magnetic tape, to storage, work well enough to handle our work. To put it another way, the bottlenecks to efficient use of computers lie elsewhere. The fact that key-to-magnetic-tape devices exist, and work properly but are not widely used would seem to confirm this contention.

Perhaps it would be charitable and timesaving not to list all such flamboyant errors. If excuses can be found, in charity, for such a speech having been tolerated, none can be found for allowing it to be reprinted.

FRED GRUENBERGER

[No Comment—Ed.]